

Producto Requerido

Documento (PRD)

LA VILLA SKATEBOARDING

Sistema de API Microservicios para E-Commerce

Cliente:

La Villa Skateboarding
Nariño, Colombia

Proveedor:

Macreat Development

Versión del Documento:

v1.0

Fecha:

01 de Septiembre de 2026

Este documento es confidencial y está destinado únicamente a las partes involucradas en el proyecto.

Índice

1. Información del Contrato	2
2. Alcance del Proyecto	2
2.1. Descripción General	2
2.2. Arquitectura	2
3. Especificación de Servicios y Endpoints	2
3.1. API Gateway (Laravel)	3
3.1.1. Autenticación	3
3.2. Servicio de Catálogo (FastAPI :9002)	3
3.2.1. Estrategia de Búsqueda Inteligente	3
3.2.2. Tablas de Base de Datos	4
3.3. Servicio de Inventario (FastAPI :9003)	4
3.3.1. Tablas	4
3.4. Servicio de Órdenes (FastAPI :9004)	4
3.4.1. Estados de Orden	4
3.4.2. Carrito	5
3.5. Servicio de Pagos (FastAPI :9005)	5
3.5.1. Tablas	5
3.6. Servicio de Notificaciones (FastAPI :9006)	5
3.6.1. Integraciones Externas	5
3.7. Servicio de Usuarios (FastAPI :9007)	5
4. Infraestructura	5
4.1. Servicios Docker	6
4.2. Volúmenes Persistidos	6
4.3. Variables de Entorno Críticas	6
5. Funcionalidades Clave	6
5.1. Tienda Pública	6
5.2. Panel Administrativo	7
5.3. Integraciones Externas	7
5.4. Búsqueda Inteligente (Actual)	7
6. Seguridad	7
7. Entregables	8
7.1. Fase 1: Diseño (Semanas 1–2)	8
7.2. Fase 2: Construcción (Semanas 3–8)	8
7.3. Fase 3: Levantamiento (Semanas 9–10)	8
7.4. Fase 4: Mantenimiento (Mes 1–12)	8
8. Cronograma	8
9. Terminos Económicos	9
10. Resumen de Endpoints	9

11.Hoja de Ruta – Funcionalidades Futuras	9
11.1. Agente Conversacional con LLM	9
11.2. Importación de Productos desde Google Drive	9
11.3. Creación Directa de Productos vía API	10
12.Ciclo de Desarrollo de Software – Modelo de Sommerville	10
12.1. Por Qué un Modelo Iterativo	10
12.2. Fases del Modelo de Sommerville	10
12.3. Aplicación en Este Proyecto	11
12.4. Ciclos de Mantenimiento	11
13.Criterios de Aceptación	11
14.Firmas	12

Información del Contrato

Campo	Detalle
Objeto	Diseño, construcción, levantamiento y mantenimiento de API microservicios para e-commerce
Valor Total	\$5.000.000 COP
Duración	12 meses(12 meses)
Moneda	Peso Colombiano (COP)
Fecha de Inicio	Por definir al momento de la firma
Entorno	Producción: VPS + Vercel

Alcance del Proyecto

Descripción General

El sistema consiste en una arquitectura de microservicios para una tienda en línea de skateboarding, compuesta por:

- **API Gateway** (Laravel/PHP) – Enrutamiento, autenticación, proxy a microservicios
- **6 Microservicios** (Python/FastAPI) – Catálogo, Inventario, Órdenes, Pagos, Notificaciones, Usuarios
- **Frontend** (Next.js/React) – Tienda pública + panel administrativo
- **Infraestructura** (Docker Compose) – PostgreSQL, Redis, MinIO, RabbitMQ, Meilisearch

Arquitectura

```

Frontend (Next.js :3000)
|
v
Gateway (Laravel :8010) ---[Sanctum Auth]---> PostgreSQL
|
+---> Catalog (:9002)      ---> PostgreSQL + MinIO
+---> Inventory (:9003)    ---> PostgreSQL
+---> Orders (:9004)      ---> PostgreSQL + Redis
+---> Payments (:9005)    ---> PostgreSQL
+---> Notifications (:9006) -> WhatsApp API + Twilio SMS
+---> Users (:9007)       ---> PostgreSQL

```

Especificación de Servicios y Endpoints

API Gateway (Laravel)

Método	Ruta	Auth
POST	/api/admin/login	No
POST	/api/admin/logout	Sanctum
GET	/api/admin/me	Sanctum
PUT	/api/admin/me	Sanctum
GET	/api/v1/catalog/{path}	No
POST	/api/v1/catalog/search	No
GET	/api/v1/{service}/health	No
POST	/api/v1/{service}/{path}	No
ANY	/api/v1/{service}/{path}	Sanctum

Autenticación

- Laravel Sanctum con tokens scoped ['admin']
- Middleware `EnsureAdmin`: verifica `user.is_admin` y `tokenCan('admin')`
- Login: email + password + device_name, retorna `plainTextToken`
- Rate limit: 5 intentos/minuto en login

Servicio de Catálogo (FastAPI :9002)

Método	Ruta	Descripción
GET	/health	Health check
GET	/products	Listar productos (parámetros: search, id)
GET	/products/{id}	Obtener producto por ID
POST	/products	Crear producto
POST	/search	Búsqueda inteligente de productos
POST	/media/presign	URL presignada para upload S3
POST	/media/{id}/complete	Marcar upload como completado
POST	/subscribers	Suscribir email
GET	/subscribers	Listar suscriptores
GET	/subscribers/count	Conteo de suscriptores

Estrategia de Búsqueda Inteligente

La búsqueda inteligente implementa una estrategia de filtrado por palabras clave con las siguientes capas:

1. **Extracción de palabras clave:** El texto libre se tokeniza y se filtran palabras vacías (español e inglés: “de”, “el”, “la”, “para”, “the”, “is”, etc.)
2. **Stemming español básico:** Se eliminan terminaciones plurales para mejorar recall (“negros” → “negro”, “ruedas” → “rueda”). Se aplica solo a palabras con 5+ caracteres para evitar distorsión de términos cortos como “tenis”
3. **Lógica OR:** Un producto coincide si contiene CUALQUIERA de las palabras clave extraídas (no todas)
4. **Scoring de relevancia:** Cada coincidencia suma 1 punto. Los resultados se ordenan por puntaje descendente, luego por nombre

5. **Campos de búsqueda:** nombre del producto, nombre de categoría, SKU, descripción
6. **Filtros duros:** categoría, subcategoría, rango de precio, marca (cuando se proporcionan como filtros estructurados)

Ejemplo: La consulta “tenis nike negros” extrae las keywords [tenis, nike, negros], aplica stemming [tenis, nike, negro], y retorna todos los productos donde CUALQUIERA de esas palabras aparece en nombre/categoría/descripción, ordenados por relevancia.

Tablas de Base de Datos

- **categories:** id, slug (unique), name, category_group
- **products:** id, name, description, sku (unique), price (Decimal 10,2), category_id (FK), media_id (FK), active
- **media:** id, storage_key (unique), bucket, mime_type, status, variants (JSON), metadata (JSON)
- **subscribers:** id, email (unique), tag

Servicio de Inventario (FastAPI :9003)

Método	Ruta	Descripción
GET	/health	Health check
GET	/inventory	Listar niveles de stock
GET	/inventory/{product_id}	Stock de un producto
PUT	/inventory/{product_id}	Establecer cantidad
POST	/inventory/{product_id}/adjust	Ajustar stock por delta
GET	/availability	Verificar disponibilidad (IDs separados por coma)

Tablas

- **stock_levels:** id, product_id (unique), sku, quantity, low_stock_threshold (default: 5)

Servicio de Órdenes (FastAPI :9004)

Método	Ruta	Descripción
GET	/health	Health check
POST	/orders	Crear orden directa
GET	/orders	Listar órdenes (skip/limit)
GET	/orders/summary	Resumen dashboard
GET	/orders/{id}	Obtener orden
POST	/orders/{id}/cancel	Cancelar orden
POST	/orders/{id}/status	Actualizar estado
GET	/cart	Obtener carrito (requiere X-Session-Id)
POST	/cart/items	Agregar item al carrito
DELETE	/cart/items/{product_id}	Eliminar item
POST	/cart/checkout	Checkout: crear orden desde carrito

Estados de Orden

pending → confirmed → shipped → delivered | cancelled

Carrito

- Almacenamiento: Redis con TTL configurable
- Clave: `cart:{session_id}`
- Checkout: convierte carrito a Orden persistente, elimina carrito

Servicio de Pagos (FastAPI :9005)

Método	Ruta	Descripción
GET	/health	Health check
POST	/payment-intents	Crear intención de pago
GET	/payment-intents	Listar intenciones
GET	/payment-intents/{id}	Obtener intención
POST	/payment-intents/{id}/confirm	Confirmar pago
POST	/payment-intents/{id}/capture	Capturar pago

Tablas

- `payment_intents`: id, order_id, amount (float), currency (default: "COP"), status (pending/confirmed/captured), provider (default: "manual")

Servicio de Notificaciones (FastAPI :9006)

Método	Ruta	Descripción
GET	/health	Health check
POST	/send-catalog	Enviar catálogo completo vía WhatsApp
POST	/send-whatsapp	Enviar mensaje WhatsApp arbitrario
POST	/notify-order	Notificar orden vía SMS (Twilio)
GET	/status	Estado de configuración WhatsApp/Twilio

Integraciones Externas

- WhatsApp Cloud API (Meta) v21.0
- Twilio SMS (fallback)
- Modo mock: responde éxito simulado si no hay API keys configuradas

Servicio de Usuarios (FastAPI :9007)

Método	Ruta	Descripción
GET	/health	Health check
GET	/users	Listar usuarios
GET	/users/{id}	Obtener usuario
POST	/users	Crear usuario (email único)
PUT	/users/{id}	Actualizar usuario

Infraestructura

Servicios Docker

Servicio	Imagen	Puerto	Propósito
PostgreSQL	postgres:16-alpine	5432	Base de datos principal
Redis	redis:7-alpine	6379	Carrito, caché
RabbitMQ	rabbitmq:3-mgmt	5672	Cola de mensajes
MeiliSearch	meilisearch:v1	7700	Motor de búsqueda
MinIO	minio/minio	9000/9001	Almacenamiento S3
Gateway	Laravel	8010	API Gateway
Catalog	FastAPI	9002	Catálogo de productos
Inventory	FastAPI	9003	Gestión de stock
Orders	FastAPI	9004	Órdenes + carrito
Payments	FastAPI	9005	Intenciones de pago
Notifications	FastAPI	9006	WhatsApp + SMS
Users	FastAPI	9007	Perfiles de usuario
Frontend	Next.js	3000	Tienda + admin

Volúmenes Persistidos

- postgres_data
- redis_data
- rabbitmq_data
- meilisearch_data
- minio_data

Variables de Entorno Críticas

Variable	Descripción
DATABASE_URL	URL de conexión PostgreSQL
NEXT_PUBLIC_API_BASE_URL	URL pública del gateway
NEXT_PUBLIC_SITE_URL	Dominio del sitio
FRONTEND_URL	Dominio para CORS
SANCTUM_STATEFUL_DOMAINS	Dominios Sanctum
ADMIN_PASSWORD	Contraseña del admin
APP_KEY	Clave de cifrado Laravel
CDN_BASE_URL	URL pública de MinIO/S3

Funcionalidades Clave

Tienda Pública

1. Listado de productos con filtrado por categoría y subcategoría
2. Búsqueda inteligente de productos por palabras clave con stemming
3. Carrito de compras persistente (sesión + Redis)
4. Checkout con creación de orden
5. Detalle de producto con imágenes
6. Sistema de suscripción por email

7. Contenido de marca: hero, editorial, heritage, identidad visual

Panel Administrativo

1. Dashboard con métricas (órdenes totales, ingresos, hoy/ayer)
2. Gestión de productos (CRUD + imágenes via presigned URLs)
3. Gestión de órdenes (estados, cancelación)
4. Gestión de inventario (stock, ajustes, alertas de bajo stock)
5. Autenticación con tokens Sanctum

Integraciones Externas

1. WhatsApp Cloud API (Meta) para notificaciones y catálogo
2. Twilio SMS para notificaciones de órdenes
3. MinIO (S3) para almacenamiento de imágenes

Búsqueda Inteligente (Actual)

- Estrategia basada en keywords con stemming español
- Categorías: decks, apparel, accessories, gear
- Subcategorías: 14 tipos (tenis, buzos, camisetas, trucks, ruedas, etc.)
- Extracción de filtros: categoría, subcategoría, precio, marca
- Lógica OR con scoring de relevancia por coincidencia
- Sin dependencia de servicios externos para búsqueda

Seguridad

- Autenticación basada en tokens (Laravel Sanctum)
- Tokens scoped con ability ['admin']
- Middleware **EnsureAdmin** en rutas protegidas
- Rate limiting: login (5/min), búsqueda NL (10/min), checkout (20/min)
- Variables de entorno para credenciales (no hardcodeadas en código fuente)
- CORS configurado por dominio
- HTTPS requerido en producción

Entregables

Fase 1: Diseño (Semanas 1–2)

- Documento de requerimientos (este documento)
- Arquitectura de microservicios
- Diseño de base de datos
- Diagrama de endpoints
- Definición de variables de entorno

Fase 2: Construcción (Semanas 3–8)

- API Gateway (Laravel) con autenticación
- 6 microservicios FastAPI
- Frontend Next.js (tienda + admin)
- Infraestructura Docker Compose
- Integración WhatsApp/SMS
- Búsqueda inteligente de productos

Fase 3: Levantamiento (Semanas 9–10)

- Configuración de VPS (Docker, nginx, TLS)
- Despliegue frontend en Vercel
- Configuración de variables de entorno producción
- Generación de APP_KEY único
- Pruebas end-to-end

Fase 4: Mantenimiento (Mes 1–12)

- Monitoreo de servicios
- Resolución de bugs
- Actualizaciones de seguridad
- Soporte técnico
- Mejoras menores (hasta 10 horas/mes)

Cronograma

Fase	Duración	Actividades
Diseño	2 semanas	PRD, arquitectura, DB, endpoints
Construcción	6 semanas	Gateway, microservicios, frontend
Levantamiento	2 semanas	VPS, Vercel, TLS, pruebas
Mantenimiento	12 meses	Monitoreo, bugs, seguridad, soporte

Terminos Económicos

Concepto	Detalle
Valor total del contrato	\$5.000.000 COP
Forma de pago	50 % al inicio, 50 % al entrega
Moneda	Peso Colombiano (COP)
Periodo de mantenimiento	12 meses
Incluye mantenimiento	Sí (hasta 10 horas/mes)

Resumen de Endpoints

Servicio	Endpoints	Descripción
Gateway	9	Enrutamiento, auth, proxy
Catálogo	10	Productos, búsqueda NL, media, suscriptores
Inventario	6	Stock, disponibilidad
Órdenes	10	Órdenes, carrito, checkout
Pagos	5	Intenciones de pago
Notificaciones	5	WhatsApp, SMS
Usuarios	5	CRUD de usuarios
Total	50	

Hoja de Ruta – Funcionalidades Futuras

Las siguientes funcionalidades están planificadas como mejoras independientes del alcance actual del contrato. Se implementarán bajo presupuesto adicional o en futuros ciclos de desarrollo.

Agente Conversacional con LLM

- **Objetivo:** Asistente virtual que responda preguntas sobre la tienda (productos, disponibilidad, tallas, precios, pedidos)
- **Tecnología:** Modelo de lenguaje (GPT, Claude, u open-source) integrado vía API
- **Alcance:** Consultas sobre catálogo, estado de órdenes, recomendaciones de productos
- **Implementación:** Microservicio independiente (`/chat`) que consume el catálogo y devuelve respuestas contextuales
- **Costo:** Requiere API key del proveedor LLM (costo operativo mensual variable)
- **Estado:** No incluido en el contrato actual. Funcionalidad futura independiente

Importación de Productos desde Google Drive

- **Objetivo:** Permitir al cliente subir un archivo CSV/Excel a Google Drive y que los productos se importen automáticamente al inventario
- **Flujo:** Google Drive API → webhook/polling → parser CSV → endpoint `/products` → registro en inventario
- **Requisitos:** Cuenta de servicio Google, permisos de lectura en Drive, estructura CSV predefinida
- **Estado:** No incluido en el contrato actual. Funcionalidad futura independiente

Creación Directa de Productos vía API

- **Objetivo:** Endpoint público o autenticado para crear productos directamente sin pasar por el panel admin
- **Endpoint:** POST `/api/v1/catalog/products` (ya existe funcionalmente, requiere autenticación)
- **Uso:** Integración con sistemas externos, imports masivos, automatización
- **Estado:** Endpoint funcional. Requiere documentación y validación de schema para uso externo

Ciclo de Desarrollo de Software – Modelo de Sommerville

El desarrollo y mantenimiento de este sistema sigue el modelo de ciclo de vida de software propuesto por **Ian Sommerville** en *Engineering Software* (10th ed.), que enfatiza la naturaleza **iterativa e incremental** del desarrollo moderno.

Por Qué un Modelo Iterativo

- Los requisitos de e-commerce evolucionan constantemente (nuevas categorías, integraciones de pago, funciones de marketing)
- La retroalimentación del usuario es esencial para priorizar mejoras
- Los microservicios permiten desplegar partes del sistema de forma independiente
- Un modelo en cascada tradicional no se adapta a los cambios continuos del negocio

Fases del Modelo de Sommerville

1. **Definición de Requisitos:** Se capturan y documentan los requisitos funcionales y no funcionales (este PRD). Se validan con el cliente antes de avanzar.
2. **Diseño del Sistema:** Se define la arquitectura (microservicios, base de datos, protocolos). Se producen diagramas de componentes, secuencia y despliegue.
3. **Implementación:** Se construyen los módulos de forma incremental. Cada microservicio se desarrolla, prueba y despliega de forma independiente.
4. **Pruebas:** Se ejecutan pruebas unitarias, de integración y end-to-end. Se validan los criterios de aceptación de cada funcionalidad.
5. **Despliegue:** Se libera incrementalmente. Primero se despliega en staging, luego en producción tras validación.
6. **Evolución:** El sistema se mantiene vivo. Se corrigen bugs, se agregan funcionalidades, se actualizan dependencias. Cada ciclo de mantenimiento repite las fases anteriores de forma reducida.

Aplicación en Este Proyecto

Principio Sommerville	Aplicación en La Villa
Desarrollo incremental	Cada microservicio se despliega y valida por separado
Refactorización continua	Se reestructura código sin cambiar funcionalidad
Pruebas automatizadas	Health checks, tests de integración, validación de endpoints
Retroalimentación del usuario	Iteraciones basadas en uso real de la tienda
Gestión de configuración	Variables de entorno, Docker Compose, despliegue reproducible
Documentación viva	Este PRD se actualiza con cada ciclo de desarrollo

Ciclos de Mantenimiento

Durante los 12 meses de contrato, el mantenimiento sigue ciclos de 2–4 semanas:

1. **Sprint de mantenimiento:** Se identifican bugs, mejoras menores y actualizaciones de seguridad
2. **Desarrollo:** Se implementan los cambios en un branch dedicado
3. **Pruebas:** Se validan los cambios contra el sistema existente
4. **Despliegue:** Se liberan los cambios en producción
5. **Revisión:** Se documentan los cambios y se actualiza este PRD si es necesario

Criterios de Aceptación

1. Todos los endpoints funcionan correctamente según especificación
2. Autenticación Sanctum operativa con tokens scoped
3. Búsqueda inteligente retorna resultados relevantes en español
4. Carrito persiste entre sesiones (Redis)
5. Checkout crea orden y limpia carrito
6. Notificaciones WhatsApp/SMS se envían correctamente
7. Panel admin muestra métricas en tiempo real
8. Imágenes se sirven correctamente vía media proxy
9. Todos los servicios pasan health check
10. Despliegue exitoso en VPS + Vercel
11. HTTPS operativo en dominio de producción
12. CORS configurado para dominio Vercel

Firmas

La Villa Skateboarding
Cliente

Nombre: _____

Fecha: _____

Macreat Development
Proveedor

Nombre: _____

Fecha: _____